The AI Open Systems Interconnection (AOSI) Reference Model: A Seven-Layer Vocabulary for Locating Performance, Correctness and Security Concerns in Generative AI Inference

Divyendu Bhatt
Independent Researcher
Divyendu.bhatt@gmail.com

9 November 2026

Abstract

A single generative AI request crosses a client application, an orchestration gateway, a retrieval pipeline, an inference-serving engine, a transformer model, a compiler and runtime stack, and accelerator hardware. Each tier has its own tools, telemetry and vocabulary, and practitioners lack a shared frame for stating *where* in this path a latency regression, a quality failure or a security threat originates. Standards, risk frameworks, industry stack diagrams, serving surveys and threat catalogues each cover part of this problem, and existing OSI-inspired AI models target agent cognition or multi-agent interoperability rather than the execution path of one request. This paper presents AOSI, an informative seven-layer reference model for generative AI inference that (i) orders the tiers from physical infrastructure (L1) to application (L7), (ii) names a canonical data state at every layer boundary, from JSON payload to silicon voltages, (iii) derives a layer-isolation method for diagnosing latency and failures, and (iv) maps the OWASP Top 10 for LLM Applications onto the layers at which each risk materialises. We position AOSI against eleven groups of related work in a structured comparison, illustrate its use on a worked (non-empirical) triage example, and state its limitations: it has not been empirically validated or adopted by any standards body. We close with a concrete evaluation plan.

Keywords: reference model, generative AI, LLM inference serving, observability, AI security, layered architecture

1 Introduction

Production generative AI is no longer a model behind an API. Serving a single chat completion with retrieval-augmented generation (RAG) [1] involves an application client, an API or AI gateway that applies policy and templates, an embedding model and vector index, a serving engine that batches requests and manages a key–value (KV) cache [2, 3], the transformer itself, a framework and compiler stack that lowers the model to kernels [4], and accelerators whose memory bandwidth often bounds decode speed [5, 6]. These tiers are usually owned by different teams, observed with different tools, and described with different words.

The consequence is a recurring practical problem: *there is no shared vocabulary for locating a failure or threat along the generative AI execution path.* A report that "the model is slow" may mean retrieval latency, queueing in the serving engine, KV-cache preemption, or memory-bandwidth saturation on the device; "the model hallucinated" may mean a retrieval miss, a stale index, a prompt-template defect, or a quantisation regression. Security controls are often specified as generic

"AI controls" without stating the tier at which the threat materialises. Sculley et al. observed that ML systems erode abstraction boundaries and accumulate system-level debt [7]; generative inference stacks inherit that problem with more tiers.

Computer networking faced an analogous problem and addressed it with the OSI Basic Reference Model [8–10]: a fixed set of layers, service boundaries between adjacent layers, and named data units at each boundary. OSI's lasting value has arguably been less its protocols than its vocabulary: engineers can say "this is a layer-2 problem" and be understood.

This paper applies the same *method*—not the OSI layers themselves—to generative AI inference. We are not the first to draw an OSI analogy for AI (Section 3); our contribution is narrower and, we argue, complementary. Specifically:

1. **A full-depth, ordered layer model of single-request generative inference** (Section 4), spanning application (L7), orchestration and gateway (L6), retrieval and knowledge (L5), inference and serving (L4), model and weight architecture (L3), framework and runtime (L2), and physical infrastructure (L1), with explicit scope boundaries and design principles.
2. **A named data state at every boundary**: JSON payload → enriched prompt → vector chunks → batched tensors → hidden states → tensor IR → silicon voltages, playing the role of OSI protocol data units.
3. **A layer-isolation diagnostic method** (Section 5) that combines a latency decomposition, per-layer trace attribution and single-variable isolation tests, including discriminators for three commonly confused causes (L5 retrieval lag, L4 KV-cache pressure, L1 memory-bandwidth saturation).
4. **A layer-indexed security mapping** (Section 6) that places each OWASP Top 10 for LLM Applications (2025) risk [11] at the layer(s) where it materialises.
5. **A structured comparison** (Section 3, Table 1) with eleven groups of related standards, frameworks and layer models, and an **evaluation plan** (Section 8) for testing the model's claims.

AOSI is a conceptual, informative model. It defines no protocol, it has not been empirically validated, and it has not been adopted by any standards development organisation. A full specification, including per-layer interfaces, metrics, failure modes and proposed telemetry attributes, is archived as a versioned technical report [12]; an IETF Internet-Draft version has been prepared for submission to the Independent Submission stream.

2 Background: Why Layered Reference Models Work

Layering as a structuring discipline predates networking. Dijkstra's THE system organised an operating system into levels, each relying only on services of the levels below [13], and Parnas argued that modules should be chosen to hide design decisions likely to change [14]. The OSI model turned these ideas into a *reference* model for an industry [8, 10]. Three properties are relevant here.

Separation of concerns with stable boundaries. Each OSI layer provides a service to the layer above through a defined boundary. Implementations of one layer can change without changing its neighbours, provided the service is preserved.

Named data units. OSI names what crosses each boundary (frames, packets, segments, and so on). A named unit can be captured, decoded and measured, which is what makes tools such as protocol analysers effective: an engineer can inspect the unit at a boundary and assert which layer misbehaved.

Reference, not implementation. ISO/IEC 7498-1 explicitly positions the model as a framework for coordinating standards, not an implementation specification [8]. The Internet's own protocols do not map one-to-one onto OSI layers, yet the vocabulary remained useful. The end-to-end argument [15] adds a caution that transfers directly to AI: some functions (e.g., correctness of the final answer) can only be fully implemented with knowledge held at the endpoints, so a layer model must not pretend that every concern is local to one layer.

Distributed tracing supplies the other ingredient AOSI relies on. Dapper showed that propagating a trace context across service boundaries makes latency in large distributed systems attributable to individual components [16]; W3C Trace Context and the OpenTelemetry generative AI semantic conventions standardise parts of this for modern stacks [17, 18]. AOSI's diagnostic method (Section 5) is essentially a proposal for *which* boundaries such traces should mark in a generative AI system.

3 Related Work

We group related work into five families and summarise the comparison in Table 1. The assessments are the author's reading of each work's public text or official description; ISO/IEC standards were assessed from their published scope statements, since full texts are not freely available. Corrections are welcome.

International standards and risk frameworks. ISO/IEC 22989 establishes AI concepts and terminology [19]. ISO/IEC 23053 provides a technology-neutral framework describing the components and functions of AI systems that use machine learning [20]. Both are normative and broad: they define *what* an AI system consists of, across the ML lifecycle, rather than how one inference request traverses a serving stack. The NIST AI Risk Management Framework [21] and its Generative AI Profile [22] define organisational risk-management functions (Govern, Map, Measure, Manage) and generative-AI-specific risks and actions; they are process frameworks rather than technical layer models. AOSI is intended to sit beneath these: it offers a technical location (e.g., "L5") at which a "Measure" or "Manage" activity can be scoped, and it defers to ISO/IEC 22989 for general AI terms.

Industry stack diagrams and MLOps architectures. The a16z reference architecture for LLM applications [23] describes data pipelines, embedding models, vector databases, orchestration, APIs and plugins, LLM caches, logging and validation, and app hosting. It is widely referenced and covers roughly AOSI L5–L7, but it treats the model as a hosted API and does not decompose serving, model, framework or hardware tiers. MLOps references—Google Cloud's continuous-delivery architecture [24] and Kreuzberger et al.'s literature- and interview-based definition of MLOps components, roles and workflows [25]—are organised around the ML *lifecycle* (training, validation, deployment, monitoring). They include model serving and monitoring components but not the internal layering of a generative inference request. AOSI excludes training (Section 4.1), which is precisely where lifecycle models apply.

Software-architecture research on foundation-model systems. Lu et al. propose a taxonomy of foundation-model-based systems [26], a pattern-oriented reference architecture for responsible FM-based systems organised into system, operation and supply-chain layers [27], and a reference architecture for FM-based agents [28]. These works treat RAG, guardrails and responsible-AI controls as architectural elements and evaluate completeness by mapping to real systems. Their unit of analysis is the software architecture and its design decisions; AOSI's unit is the execution path of a request down to hardware, with named boundary states. The two are complementary: a Lu et al. guardrail pattern would be located at AOSI L6, a RAG pattern at L5.

Serving-systems research and surveys. Systems work such as Orca's iteration-level scheduling [2], PagedAttention [3], chunked prefill [29] and prefill/decode disaggregation [30] shows that the serving tier has its own bottlenecks, distinct from model architecture and hardware. Surveys organise this space: Miao et al. span algorithmic and system techniques [31]; Li et al. group recent advances into KV-cache and memory management, computation optimisation, cloud deployment and emerging areas including RAG and mixture-of-experts [32]; Pan and Li structure inference systems from request-processing operators to memory management and multi-replica designs [33]; Yuan et al. use the roofline model [5] to explain why decoding is often memory-bound [6]. MLPerf Inference provides system-level benchmarks [34]. These works are taxonomies of *optimisation techniques* or measurements, not reference models with layer boundaries and boundary states; AOSI draws on them as evidence that L4, L3, L2 and L1 deserve separate layers.

OSI-inspired AI layer models. At least two prior works use the OSI analogy explicitly. The ACE framework of Shapiro et al. is a six-layer cognitive architecture for autonomous agents, from an Aspirational Layer to Task Prosecution, explicitly inspired by OSI [35]. Rodrigo-Ginés proposes a seven-level OSI-inspired interoperability stack for agentic AI, from physical infrastructure and runtime execution to agent-to-agent communication, coordination workflows, and governance and trust [36]. ACE layers an agent's reasoning and control; Rodrigo-Ginés layers interoperability between agents. Neither decomposes the inference path within a single request or defines boundary data states for it. AOSI's reserved Layer 8 (agentic workflows; Section 8) is where these models would connect.

AI security catalogues. The OWASP Top 10 for LLM Applications (2025) [11] ranks risk categories; MITRE ATLAS catalogues adversary tactics and techniques against AI-enabled systems [37]; NIST AI 100-2 E2025 provides a taxonomy and terminology of adversarial ML attacks and mitigations [38]; Google's SAIF frames security practices for AI systems [39]. These describe *what* can go wrong and what to do; none of them is organised by the execution layer at which a threat materialises. Section 6 uses AOSI as such an index.

Positioning. Table 1 shows the gap AOSI targets. Works that reach the hardware either layer agents (Rodrigo-Ginés) or catalogue optimisations (serving surveys); works that cover RAG and guardrails stop at the model API; security catalogues are not layer-indexed; and none defines a named data state at every boundary. AOSI's claim is therefore *not* that it is the first layered or OSI-style model for AI, but that it combines (a) full-depth coverage of one request's execution path, (b) named boundary states, (c) a diagnostic method built on them and (d) a layer-indexed security mapping. Whether this combination is useful is an empirical question we have not yet answered (Section 8.2).

Table 1: Comparison of AOSI with related standards, frameworks and layer models. ● = explicit, distinct element; ◐ = discussed but not as a distinct layer or boundary; ○ = not a focus. "States" = a named data representation at each inter-layer boundary. "Diag." = a method for attributing latency or failures to a tier; "Sec." = security threats or controls addressed (layer-indexed only in AOSI); "HW" = hardware layer; "Serve" = inference-serving layer. Assessments reflect the author's reading of public descriptions (Section 3).

Work	Primary scope	HW	Serve	RAG	States	Diag.	Sec.	Nature
ISO/IEC 22989 [19]	AI concepts & terminology	○	○	○	○	○	◐	Normative std.
ISO/IEC 23053 [20]	ML system components & functions	○	◐	○	○	○	○	Normative std.
NIST AI RMF / 600-1 [21, 22]	Organisational AI risk process	○	○	○	○	○	◐	Voluntary guidance
a16z LLM app stack [23]	LLM application tier (≈L5–L7)	○	◐	●	○	○	◐	Industry, inform.
MLOps refs. [24, 25]	ML lifecycle (CI/CD/CT)	◐	◐	○	○	◐	○	Industry / academic
Lu et al. [27, 28]	FM-based system & agent design patterns	○	◐	●	○	○	◐	Academic, inform.
Serving surveys [31–33]	Serving optimisation techniques	◐	●	◐	○	○	○	Academic surveys
ACE [35]	Agent cognition, 6 layers	○	○	○	○	◐	○	Academic, inform.
Rodrigo-Ginés [36]	Multi-agent interoperability, 7 levels	●	◐	○	○	○	◐	Position paper
OWASP LLM Top 10 [11]	LLM application risk list	○	◐	●	○	○	●	Community, inform.
MITRE ATLAS [37]	Adversary TTP knowledge base	○	◐	◐	○	○	●	Community, inform.
AOSI (this work)	**Single-request GenAI inference, L1–L7**	●	●	●	●	●[a]	●[a]	**Informative proposal**

[a] Proposed method and mapping; not yet empirically validated (Section 8).

4 The AOSI Model

4.1 Scope

AOSI covers *inference-time* execution of generative models—primarily autoregressive transformer language models and, by extension, multimodal models—from client request to device execution and back. Training and fine-tuning pipelines, data governance upstream of the retrieval index, wire protocols, API schemas and certification are out of scope. Named products are illustrative occupants of a layer, not endorsements.

4.2 Design principles

AOSI's layer definitions are governed by seven principles: (P1) single primary responsibility per layer, with cross-layer components described once per layer they occupy; (P2) a named data state at every boundary; (P3) adjacency of interfaces, with any cross-layer shortcut documented as an explicit coupling; (P4) observability by construction, with telemetry emitted at boundaries and attributable to one layer; (P5) technology neutrality; (P6) awareness of a cost gradient—latency and computational cost increase toward lower layers, while semantic abstraction and policy surface increase toward upper layers; and (P7) security controls placed at the layer where a threat materialises, with defence-in-depth at adjacent layers where justified.

4.3 Layers

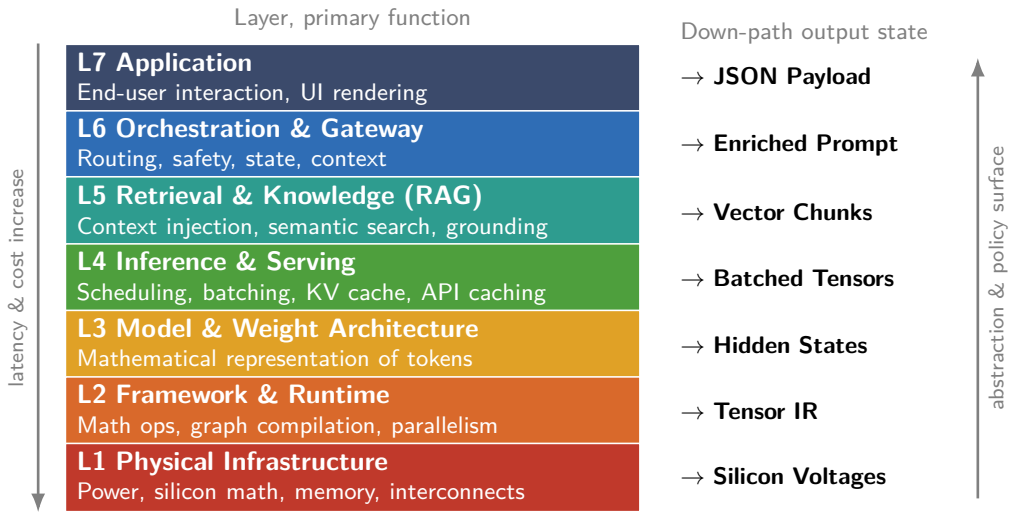


Figure 1: The seven AOSI layers and the data state each layer emits on the down-path (request). The up-path (response) returns logits from L3 to L4 for sampling, and detokenised text through L6 output policy to L7. L5 is normally traversed only on the down-path.

Figure 1 shows the stack. We summarise each layer by scope and characteristic failure modes; full component lists, interfaces and metrics are in the technical report [12].

L7 Application. User interaction, request serialisation and rendering of streamed output (chat UIs, IDE assistants, SDKs; SSE or WebSocket streaming). Failure modes: client timeouts shorter than server SLOs, proxies that buffer and defeat streaming, retry storms that amplify load below, rendering stalls misattributed to the model.

L6 Orchestration & Gateway. The policy enforcement point: authentication, quotas, model routing, prompt and chat templates, session memory, input and output guardrails, and invocation of L5. Failure modes: prompt bloat from unbounded history, guardrail latency dominating time-to-first-token (TTFT), output moderation that buffers the stream, template mismatch with the model's chat format, prompt injection passing policy checks.

L5 Retrieval & Knowledge. Query embedding, approximate-nearest-neighbour search, hybrid retrieval, ACL filtering and reranking over the index *as served* [1]. Failure modes: retrieval lag (non-resident index, remote hops), low recall from embedding or chunking mismatch, stale indexes, cross-tenant ACL leakage, poisoned documents.

L4 Inference & Serving. Tokenisation, admission control, iteration-level scheduling [2], KV-cache management [3], prefix caching, chunked prefill [29], disaggregation [30], speculative and constrained decoding. Failure modes: KV-cache exhaustion leading to preemption and time-per-output-token (TPOT) spikes, head-of-line blocking from long prefills, over-batching beyond SLO, low prefix-cache hit rates caused by non-deterministic L6 prefixes.

L3 Model & Weight Architecture. The model as mathematics: embeddings, attention variants, MLP or mixture-of-experts blocks, normalisation, output head, and the numerical format of weights, activations and KV cache. Failure modes: numerical instability in low precision, quality regression from quantisation, expert load imbalance, context-length extrapolation failure, configuration mismatch (e.g., positional-encoding scaling).

L2 Framework & Runtime. Lowering of the model program to kernels and collective schedules: frameworks, graph compilers, kernel libraries (e.g., IO-aware attention kernels [4]), device runtimes and collective-communication libraries. Failure modes: graph breaks and recompilation jitter, launch-bound execution at small batch, collective stalls from topology-unaware placement, allocator fragmentation.

L1 Physical Infrastructure. Accelerators, memory hierarchy (HBM, SRAM, host memory), intra- and inter-node interconnects, power and cooling. Failure modes: memory-bandwidth saturation during decode [6], thermal or power throttling, link degradation, hardware errors, noisy neighbours under device partitioning.

4.4 The data-state chain

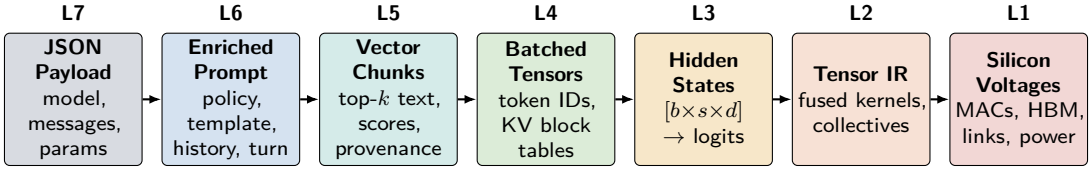


Figure 2: The canonical down-path data-state chain. Each state is the representation of a request as it leaves a layer and is the natural capture point for boundary telemetry.

Principle P2 requires a named data state at every boundary (Figure 2). On the down-path: L7 emits a **JSON payload** (e.g., model, messages, tools, sampling parameters, stream flag); L6 emits an **enriched prompt**—an ordered sequence of system policy, template, session state, grounding slot and user turn, with routing decision and policy verdicts attached; L5 returns **vector chunks**—ranked retrieved passages with scores and provenance, merged by L6 into the prompt; L4 forms **batched tensors**—token and position IDs, attention metadata, KV-cache block

tables and batch-slot assignments; L3 produces **hidden states**—per-block activations and, finally, logits; L2 lowers the computation to **tensor IR**—captured or compiled graphs, fused kernels and collective schedules; and L1 executes it as **silicon voltages**—the physical switching behind multiply–accumulate operations, memory transactions and link signalling, observable as utilisation, bandwidth and power. The last label is deliberately physical shorthand: what is observable at L1 is switching activity expressed as hardware counters, memory and link throughput, power and temperature, not voltages as such.

Two properties make the chain useful rather than decorative. First, each state has a measurable size that propagates downward: enriched-prompt token count at L6/L4 determines prefill work at L3–L1; k and chunk length at L5/L6 determine part of that token count; batch composition at L4/L3 determines the arithmetic intensity seen at L1. Second, each state is a *checkpoint for blame*: if the state entering a layer is unchanged and the layer's span grew, the layer itself is implicated; if the incoming state changed, the layer above is implicated (Section 5.4).

Request lifecycle. On the down-path, L6 authenticates, routes, applies guardrails and templates, calls L5, and hands the enriched prompt to L4, which tokenises, checks the prefix cache, allocates KV blocks and admits the sequence into a batch; L3–L1 execute the prefill forward pass. The first sampled token fixes TTFT. Each subsequent token is a decode iteration that reads weights and the KV cache from device memory; at typical batch sizes decode tends to be memory-bandwidth-bound [6], so TPOT is governed largely by L4 batching and KV behaviour and by L1 memory bandwidth. Tokens are detokenised at L4 and stream through L6 output policy to L7.

5 Cross-Layer Diagnostics

5.1 Principle

A protocol analyser makes network faults tractable by decoding a captured unit layer by layer. AOSI proposes the analogous discipline for generative AI: *each request should produce a trace in which every span is attributed to exactly one AOSI layer, and each boundary hand-off is timestamped.* Concretely, an AOSI-aligned deployment propagates one trace context from L7 through L4 [16, 17]; tags each span with a layer attribute alongside OpenTelemetry generative AI attributes [18]; exports serving-engine and device metrics on a common time base with node, device and replica labels so that spans can be joined to device telemetry; and captures framework-level profiles on demand.

5.2 Latency decomposition

For a streamed response with N_{out} output tokens,

$$E2E \approx \text{TTFT} + (N_{\text{out}} - 1) \cdot \text{TPOT}, \tag{1}$$

$$\text{TTFT} \approx t_{L7} + t_{L6}^{\text{gw}} + t_{L6}^{\text{guard,in}} + t_{L5} + t_{L4}^{\text{queue}} + t_{L4}^{\text{tok}} + t_{\text{prefill}}^{L3\text{–}L1} + t_{\text{up}}, \tag{2}$$

$$\text{TPOT} \approx t_{L4}^{\text{sched}} + t_{\text{decode}}^{L3\text{–}L1} + t_{L6}^{\text{guard,out}}. \tag{3}$$

The decomposition halves the search space. TTFT inflated with TPOT normal points *above* L4 compute (L6 overhead, L5 retrieval, L4 queueing, or longer prompts). TPOT inflated with TTFT normal points at or *below* L4 decode (KV pressure, batch growth, memory bandwidth, throttling, collectives) or at buffering output guardrails in L6. Both inflated suggests systemic causes: overload, device degradation, or retry storms.

5.3 Discriminating three commonly confused causes

Table 2 lists signals that separate L5 retrieval lag, L4 KV-cache pressure and L1 memory-bandwidth saturation, each of which can surface as "the model is slow". The signatures follow from the mechanisms described in the serving and roofline literature [3, 6]; they are hypotheses to be calibrated per deployment, not measured thresholds.

Table 2: Discriminating signals for three causes of perceived slowness. Signatures are derived from mechanism, not measurement, and must be calibrated per deployment.

	L5 retrieval lag	**L4 KV-cache pressure**	**L1 memory-bandwidth saturation**
Latency signature	TTFT up; TPOT unchanged	TPOT up and spiky; TTFT up if admission blocks	TPOT up smoothly with batch and context
Trace location	L5 span grows before any L4 span	L4 queue span grows; decode shows preemption gaps	L4 decode span grows uniformly per step
Engine / device signals	Engine and device normal	KV utilisation near capacity; preemption counters rise	Memory-bandwidth utilisation pinned near ceiling; compute pipes comparatively idle
Isolation test	Replay with retrieval bypassed and an equal-length fixed context	Replay with reduced concurrency or larger KV budget	Fixed-batch sweep of context length or batch size
Remediate at	Index residency, co-location, smaller k	Prefix caching, KV quantisation, admission limits, chunked prefill	Weight/KV quantisation, speculative decoding, scale-out

5.4 Triage procedure

Diagnosis proceeds top-down for attribution and bottom-up for confirmation: (1) quantify the symptom (TTFT, TPOT, E2E, error rate, goodput or quality; percentile and window); (2) decompose it into TTFT and TPOT movement; (3) walk the trace top-down (L7→L4) against a baseline—the first layer whose span distribution shifted is the *candidate layer*; (4) check the boundary state entering that layer (e.g., prompt tokens at L6/L4, k at L5/L6, batch size at L4/L3), since a changed input implicates the layer above; (5) correlate with lower-layer telemetry (engine metrics, framework profiles, device counters); (6) run the candidate layer's isolation test, changing exactly one variable; (7) confirm bottom-up that lower layers are healthy under the test; (8) remediate at the owning layer and re-measure the original symptom; and (9) record the incident with its AOSI layer attribution. Step 9 is what would make AOSI testable at scale: a corpus of layer-attributed incidents is the dataset our evaluation plan requires (Section 8.2).

6 Security Mapping

Under principle P7, each threat is placed at the layer where it materialises. Table 3 maps the OWASP Top 10 for LLM Applications (2025) [11] onto AOSI. The mapping is the author's; it is intended to be contested and refined. Two observations follow. First, several risks are multi-layer by nature: prompt injection is *enforced against* at L6 but *arrives* via L7 (direct) or L5 (indirect, through retrieved content [40]); misinformation is the joint outcome of grounding (L5), model behaviour (L3) and presentation (L7). Second, a layer index exposes control gaps that a flat list hides: an architecture whose only control for LLM04 and LLM08 is an L6 guardrail has no control

at the layer (L5 or L3) where poisoning materialises. The same index can be applied to MITRE ATLAS techniques [37] and the NIST adversarial ML taxonomy [38]; we leave that mapping to future work.

Table 3: OWASP Top 10 for LLM Applications (2025) mapped to the AOSI layer(s) where each risk materialises, with representative layer-local controls. Author's mapping; illustrative, not normative.

OWASP risk (2025)	**Primary layer(s)**	**Representative controls at that layer**
LLM01 Prompt Injection	L6 (arrives via L7 or L5)	Separation of system, user and retrieved content; input classifiers; least-privilege tool scopes; L5 content scanning
LLM02 Sensitive Information Disclosure	L6, L5, L3	Output filtering and redaction (L6); ACL-filtered retrieval (L5); training-data extraction testing (L3)
LLM03 Supply Chain	L2, L3	Pinned, verified dependencies and SBOMs (L2); signed, hash-verified model artefacts and provenance (L3)
LLM04 Data and Model Poisoning	L3 (weights), L5 (corpus)	Model provenance and red-teaming (L3); ingestion integrity and provenance checks (L5)
LLM05 Improper Output Handling	L6, L7	Schema validation (L6); rendering that never executes untrusted content (L7)
LLM06 Excessive Agency	L6 (future L8)	Tool permission scoping, human approval gates, action budgets
LLM07 System Prompt Leakage	L6	Secrets kept out of prompts; leakage detection on output
LLM08 Vector and Embedding Weaknesses	L5	Vector-store access control; tenant isolation; encryption at rest
LLM09 Misinformation	L5, L3, L7 (cross-layer)	Grounding evaluation and freshness (L5); quality evaluation (L3); citation display (L7)
LLM10 Unbounded Consumption	L4, L6	Token and time budgets (L4); rate limits and quotas (L6)

Beyond OWASP, AOSI surfaces threats that application-level lists tend not to emphasise: cross-request leakage through shared prefix or KV caches (L4), unsafe deserialisation of model files and compromised kernels or compilers (L2), and hardware side channels or memory residue between tenants (L1). An AOSI-aligned description of a system should state, per layer, which threats are in scope and which controls apply, and should not rely on a single layer as the sole control for threats that materialise at another.

7 Illustrative Worked Example

Note. This section is a constructed, hypothetical walk-through intended only to show how the model's vocabulary and procedure are applied. It does not describe a real incident and reports no measurements.

Setting. Consider an internal question-answering assistant: a web client (L7) calls an AI gateway that applies a system prompt, conversation history and guardrails (L6); the gateway retrieves policy documents from a vector index (L5); a self-hosted serving engine with continuous batching and paged KV cache (L4) runs a quantised transformer (L3) compiled with a standard framework (L2) on a multi-GPU node (L1).

Symptom A: "the assistant got slow this week". Step 1 restates the complaint as a metric: suppose the operators observe that p95 TTFT has risen relative to the previous week while TPOT has not. Step 2 therefore directs attention above L4 compute. Walking the trace (Step 3), they

compare per-layer span distributions; suppose the L5 span is unchanged but the L4 queue and prefill spans have grown. Step 4 checks the boundary state at L6/L4 and finds that enriched-prompt token counts have grown, because a recent L6 change now replays full conversation history instead of a summary. The input to L4 changed, so the layer *above* is implicated: the root cause is at L6, even though the time is spent at L4–L1. The isolation test (Step 6) replays the same requests with history truncated; if TTFT returns to baseline, the attribution is confirmed. The remediation is at L6 (history windowing or summarisation), not at L4 (adding replicas), which would only have masked the cause. The incident is recorded as *L6, prompt growth, manifesting at L4.*

Symptom B: "the assistant cites the wrong policy". Latency is normal, so the decomposition is uninformative and the operators move to quality signals. The AOSI boundary states structure the questions: were the *vector chunks* returned at L5/L6 the right documents (a retrieval-evaluation question at L5)? Did they reach the *enriched prompt* intact and in the intended position (an L6 template question)? Given correct context, did the model use it (an L3 question, testable by replaying the identical enriched prompt against a reference-precision model)? Each question isolates one layer by holding the incoming state fixed. If, say, the index had not been refreshed after a policy update, the fault is an L5 freshness failure, which also corresponds to OWASP LLM09 and is mitigated at L5 rather than by a stricter L6 output filter.

What the example shows and does not show. The example demonstrates that the vocabulary distinguishes *where time is spent* from *where the cause lies*, and that boundary states give a principled order of questions. It does not show that practitioners using AOSI diagnose faster or more accurately than without it; that is the hypothesis our evaluation plan tests.

8 Discussion, Limitations and Evaluation Plan

8.1 Limitations and threats to validity

- **No empirical validation.** The layer boundaries, discriminating signals and security mapping are argued from mechanism and the literature, not measured. Thresholds in the technical report are explicitly illustrative.
- **No standards adoption.** AOSI is a single-author proposal. It has not been reviewed or adopted by ISO/IEC, ITU-T, IETF, NIST or any other body.
- **Construct validity of boundaries.** Real systems violate strict layering: serving engines ship their own kernels (L4/L2), gateways cache responses (L6 behaving like L4), and embedding models are served by L4 engines inside L5. AOSI handles these through principle P1 (describe a component once per layer it occupies) and P3 (document couplings), but whether this remains workable for real inventories is untested.
- **Ordering of L5.** Retrieval is placed between orchestration and serving because it transforms the prompt before serving; it is, however, invoked by L6 and is not on the up-path. Alternative placements (e.g., retrieval as a peer service of L6) are defensible.
- **Scope.** Training, fine-tuning and data pipelines are excluded; agentic, multi-step and multi-agent workflows are reserved for future layers. Many current production issues arise exactly there.
- **Author bias.** The comparison in Table 1 and the mapping in Table 3 were produced by the model's author. Independent replication of both is needed.

- **Rapid technological change.** Named components will date quickly. The model's claim is that the *layers and boundary states* are stable even as occupants change; this is itself a hypothesis.

8.2 Evaluation plan

We propose three studies, ordered by cost.

1. **Incident classification study.** Collect publicly available post-incident reports and issue-tracker reports from open-source serving engines, gateways and RAG frameworks. Two or more independent raters classify each by AOSI layer of *root cause* and of *manifestation*, using a written codebook. Outcomes: inter-rater agreement (e.g., Cohen's or Fleiss' κ), the fraction of reports that cannot be assigned to a layer (a direct test of coverage), and the frequency with which root cause and manifestation layers differ (a test of the claim that the distinction matters).
2. **Practitioner survey and card-sorting.** Ask practitioners across application, platform, ML and infrastructure roles to sort component and symptom cards into layers, and to rate the clarity and usefulness of each boundary. Outcomes: agreement with AOSI's assignments, boundaries with systematic disagreement, and candidate revisions.
3. **Controlled diagnosis experiment.** Inject single, known faults (e.g., retrieval delay, KV-budget reduction, prompt growth, power capping) into an instrumented open-source stack, and compare time-to-correct-attribution and accuracy for participants with and without AOSI-tagged traces. This is the only study that would test the model's central practical claim.

Results from any of these studies, including negative results, would warrant revision of the model.

8.3 Future layers

Three extensions are reserved and informative: an L8 for agentic workflows (planning, tool use, multi-agent coordination), where ACE [35], Lu et al.'s agent architecture [28] and Rodrigo-Ginés's interoperability stack [36] are natural points of alignment; an L9 or cross-cutting plane for distributed and federated inference across sites; and a global optimisation plane relating per-layer controls to end-to-end goals such as energy per output token. An open question is whether these are layers at all, or planes that cut across L1–L7.

9 Conclusion

Generative AI inference spans tiers that are engineered, observed and secured separately, and the field lacks a shared way to say where a problem lives. AOSI borrows OSI's method—ordered layers, stable boundaries and named data units—and applies it to the execution path of a single generative request, from silicon to application. Its specific contribution is the combination of full-depth layering, a named data state at every boundary, a layer-isolation diagnostic method built on those states, and a layer-indexed security mapping; it does not claim to be the first layered or OSI-inspired model for AI. The model is unvalidated, and its value depends on whether practitioners can apply it consistently and whether it shortens diagnosis. We have outlined studies to test both, and we invite counter-examples and alternative boundaries.

Availability

The full specification (version 1.2.1), including per-layer interfaces, metrics, failure modes, a diagnostic matrix and proposed telemetry attributes, is archived at `https://doi.org/10.5281/zenodo.24073122` [12] and maintained at `https://github.com/CodingThrill0012/aosi-model` under CC-BY-4.0.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*, 2020. URL `https://arxiv.org/abs/2005.11401`.

[2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022. URL `https://www.usenix.org/conference/osdi22/presentation/yu`.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL `https://arxiv.org/abs/2205.14135`.

[5] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[6] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024. URL `https://arxiv.org/abs/2402.16363`.

[7] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503–2511, 2015.

[8] ISO/IEC. ISO/IEC 7498-1:1994, Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model. International Organization for Standardization, 1994. URL `https://www.iso.org/standard/20269.html`.

[9] ITU-T. Recommendation X.200: Information technology – Open Systems Interconnection – Basic Reference Model: The basic model. International Telecommunication Union, 1994. URL `https://www.itu.int/rec/T-REC-X.200`.

[10] Hubert Zimmermann. OSI reference model – the ISO model of architecture for open systems interconnection. *IEEE Transactions on Communications*, 28(4):425–432, 1980. doi: 10.1109/TCOM.1980.1094702.

[11] OWASP GenAI Security Project. OWASP Top 10 for LLM Applications 2025, 2025. URL `https://genai.owasp.org/llm-top-10/`.

[12] Divyendu Bhatt. AOSI-RFC-0001: The AI Open Systems Interconnection (AOSI) Reference Model: A 7-Layer Stack for Generative AI Execution. Zenodo, version 1.2.1, 2026.

[13] Edsger W. Dijkstra. The structure of the "THE"-multiprogramming system. *Communications of the ACM*, 11 (5):341–346, 1968. doi: 10.1145/363095.363143.

[14] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[15] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984. doi: 10.1145/357401.357402.

[16] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Technical Report dapper-2010-1, Google, 2010. URL `https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/`.

[17] W3C. Trace Context. W3C Recommendation. URL `https://www.w3.org/TR/trace-context/`.

[18] OpenTelemetry. Semantic Conventions for Generative AI Systems. URL `https://opentelemetry.io/docs/specs/semconv/gen-ai/`. Accessed 9 November 2026.

[19] ISO/IEC JTC 1/SC 42. ISO/IEC 22989:2022, Information technology – Artificial intelligence – Artificial intelligence concepts and terminology. International Organization for Standardization, 2022. URL https://www.iso.org/standard/74296.html.

[20] ISO/IEC JTC 1/SC 42. ISO/IEC 23053:2022, Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). International Organization for Standardization, 2022. URL https://www.iso.org/standard/74438.html.

[21] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, 2023.

[22] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. Technical Report NIST AI 600-1, NIST, 2024.

[23] Matt Bornstein and Rajko Radovanovic. Emerging Architectures for LLM Applications. Andreessen Horowitz, 2023. URL https://a16z.com/emerging-architectures-for-llm-applications/.

[24] Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning. Cloud Architecture Center. URL https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning. Accessed 9 November 2026.

[25] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *arXiv preprint arXiv:2205.02302*, 2022. URL https://arxiv.org/abs/2205.02302.

[26] Qinghua Lu, Liming Zhu, Xiwei Xu, Yue Liu, Zhenchang Xing, and Jon Whittle. A taxonomy of foundation model based systems through the lens of software architecture. *arXiv preprint arXiv:2305.05352*, 2023. URL https://arxiv.org/abs/2305.05352.

[27] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, and Jon Whittle. A reference architecture for designing foundation model based systems. *arXiv preprint arXiv:2304.11090*, 2023. URL https://arxiv.org/abs/2304.11090.

[28] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, Stefan Harrer, and Jon Whittle. Towards responsible generative AI: A reference architecture for designing foundation model based agents. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, pages 119–126, 2024. doi: 10.1109/ICSA-C63560.2024.00028.

[29] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023. URL https://arxiv.org/abs/2308.16369.

[30] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024. URL https://arxiv.org/abs/2401.09670.

[31] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023. URL https://arxiv.org/abs/2312.15234.

[32] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. LLM inference serving: Survey of recent advances and opportunities. *arXiv preprint arXiv:2407.12391*, 2024. URL https://arxiv.org/abs/2407.12391.

[33] James Pan and Guoliang Li. A survey of LLM inference systems. *arXiv preprint arXiv:2506.21901*, 2025. URL https://arxiv.org/abs/2506.21901.

[34] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, et al. MLPerf inference benchmark. *arXiv preprint arXiv:1911.02549*, 2019. URL https://arxiv.org/abs/1911.02549.

[35] David Shapiro, Wangfan Li, Manuel Delaflor, and Carlos Toxtli. Conceptual framework for autonomous cognitive entities. *arXiv preprint arXiv:2310.06775*, 2023. URL https://arxiv.org/abs/2310.06775.

[36] Francisco-Javier Rodrigo-Ginés. Toward an OSI-Inspired Stack for Agentic AI: From Fragmented Frameworks to Open Standards. Zenodo, 2025.

[37] The MITRE Corporation. ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems. URL https://atlas.mitre.org/. Accessed 9 November 2026.

[38] Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. Technical Report NIST AI 100-2 E2025, NIST, 2025.

[39] Google. Secure AI Framework (SAIF). URL https://saif.google/. Accessed 9 November 2026.

[40] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL https://arxiv.org/abs/2302.12173.